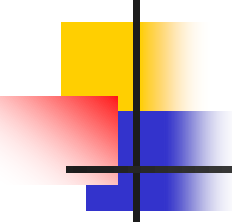
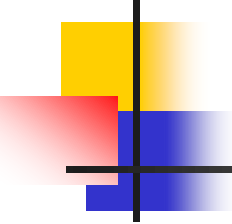


Declarative prototyping



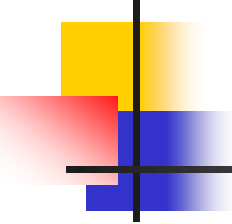
Declarative prototyping

- We present a simple programs development methodology based on mathematical induction, declarative prototyping, procedural design and implementation (the references for this chapter include [Boe84,Mit96,Pie18,RL99,Som15,Zav89]).
- We use the functional programming language Haskell [PJH99] for declarative prototyping, and C as a language for procedural implementation.
- For comparison purposes, we also present Scala [OSV16] solutions.



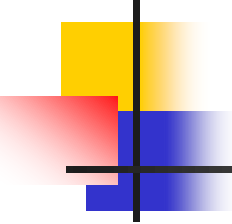
Declarative prototyping

- Haskell is a lazy purely functional programming language named after the famous logician Haskell Curry, whose contributions to lambda calculus and combinatory logic are well-known [CF58]
- Classic examples of high-level languages that can be used for prototyping purposes include: Lisp, Prolog and Smaltalk.



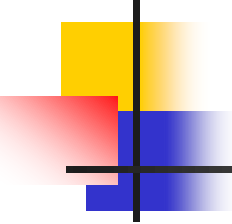
Declarative prototyping

- Haskell is a modern strongly typed functional programming language, appropriate for prototypes development.
- Advantages of Haskell as a prototyping tool:
 - Declarative specifications
 - Referential transparency (it provides support for equational reasoning)
 - Polymorphism and higher-order functions
 - A Haskell specification is typically much shorter than a corresponding C implementation



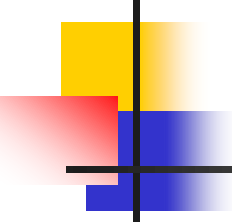
Declarative prototyping

- Haskell programs can be seen as 'executable mathematics' [RL99].
- Alternatively, we could adopt Z [Spi92] and develop formal specifications. Z specifications are more abstract, but are not executable.
- Haskell prototypes are executable and, therefore, can easily be evaluated and tested.
- It is generally accepted that prototyping reduces the number of problems with the requirements specifications [Boe84,Som15].
- The approach considered in this chapter is useful when the problems are novel or difficult.



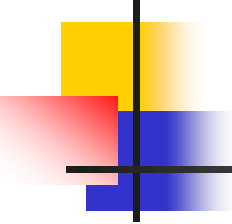
Declarative prototyping

- Scala [OSV2016] is a strongly typed multiparadigm language providing support for functional, object oriented (OO), imperative and concurrent programming
 - Data structures can be designed as classes in OO style
 - Computation can be expressed both in imperative (Java like) style and in functional style



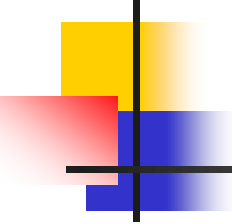
Declarative prototyping

- Scala is an OO language in pure form
 - Every value is an object
 - Every computation is implemented as a method
- With full support for functional programming, including
 - First class and higher order functions
 - Pattern matching (with case classes)
- Scala programs compile to JVM bytecodes
 - The performance of imperative style Scala programs is on par with corresponding Java programs
 - The two languages interoperate seamlessly



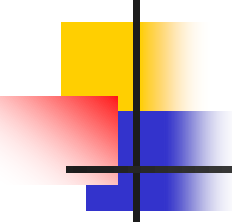
Declarative prototyping

- We present a **methodology** involving the following steps:
 1. *Build a Haskell declarative specification (prototype)*
The prototype is built by an inductive reasoning, which proves the correctness of the specification. For some examples we also present Scala solutions
 2. *Design a procedural solution* This step involves procedural design decisions, decisions concerning data structures representation, memory allocation policies, etc.
 3. *Accomplish the procedural implementation* We will use C for procedural implementation.



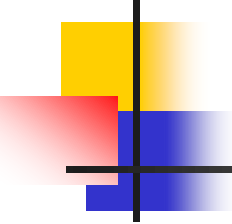
Declarative prototyping

- **Mathematical induction** is a convenient tool for recursive functions design (for the functions defined on finite structures).
- The most common forms of induction are
 - Induction on natural numbers
 - Structural induction
- They can be treated as instances of a general form of induction, called *well-founded induction* (see e.g. [Mit96]).



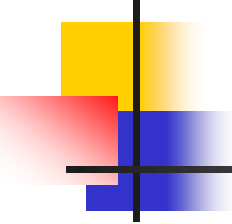
Declarative prototyping

- A *well-founded relation* on a set A is a binary relation $\prec$ on A with the property that there is no infinite descending sequence $a_0 \succ a_1 \succ a_2 \succ \dots$
 - A well-founded relation need not be transitive (example: $i \prec j$ if $j = i+1$, on the natural numbers).
 - A well-founded relation can not be reflexive (if $a \prec a$ then there is an infinite descending sequence $a \succ a \succ a \succ \dots$)
- An equivalent definition is that a binary relation $\prec$ on A is well-founded iff every nonempty subset B of A has a minimal element, where $a \in B$ is *minimal* if there is no $a' \in B$ with $a' \prec a$.



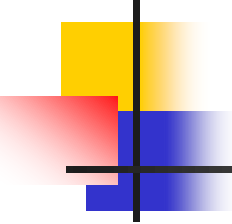
Declarative prototyping

- (Generalized or) Well-founded induction principle
 - Let $\prec$ be a well-founded binary relation on set A and let P be some property on A . If $P(a)$ holds whenever we have $P(b)$ for all $b \prec a$, then $P(a)$ is true for all $a \in A$.
- More familiar forms of induction can be obtained by using the following well-founded relations:
 - $m \prec n$ if $m+1=n$, for natural number induction
 - $e \prec e'$ if e is an immediate sub-expression of e' , for structural induction



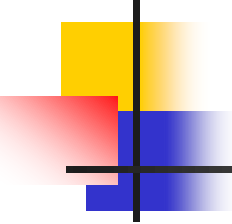
Declarative prototyping

- In the sequel we will use mathematical induction to prove the correctness of recursive definitions.
- In each case, we will define a *complexity measure*: a function that maps the concrete structures in the problem domain to a set equipped with a well-founded relation.
 - The complexity measure must be chosen so that it decreases upon any recursive call.
- We will present various kinds of inductive reasoning.



Declarative prototyping

- For simplicity, we do not consider Haskell specifications based on higher-order mappings, and we only give recursive C implementations.
- Haskell is polymorphic. C is monomorphic. A Haskell prototype can specify an entire class of C implementations.
 - For simplicity, we ignore this aspect, and we only consider data structures containing primitive types (numeric values).



Declarative prototyping

- **Haskell → C transcription**

- Each Haskell function in the declarative specification is translated to a corresponding C function in the procedural implementation (using auxiliary C functions if necessary).
- Haskell functions defined by multiple equations are implemented using conditional statements in C.
- For each recursive call in the Haskell specification there is a corresponding recursive call in the C implementation.



Declarative prototyping

- **Example Set union**

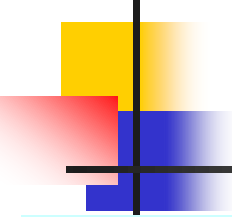
- Haskell specification:

```
member :: (Int,[Int]) -> Bool
member (e,[]) = False
member (e,x:xs) = if (e == x) then True else member (e,xs)

union :: ([Int],[Int]) -> [Int]
union ([],ys) = ys
union (x:xs,ys) = if member(x,ys) then union(xs,ys)
                  else x:union(xs,ys)
```

- The specification is correct. This follows by induction on a **simple complexity measure**:

- `member(e,xs)` - by induction on `length(xs)` (/ structural induction)
- `union(xs,ys)` – by induction on the `length(xs)`, assuming that `xs` and `ys` are lists without duplicated elements

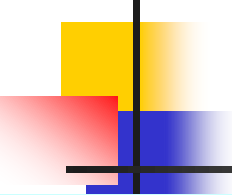


Declarative prototyping

- The Haskell prototype behaves as follows (experiments performed using the Hugs interpreter):

```
Main> union([], [1,2,3])  
[1,2,3]
```

```
Main> union([6,7,5,3], [5,6,9,1,2,7])  
[3,5,6,9,1,2,7]
```

Declarative prototyping

- Haskell also provides
 - Higher order functions (which enable partial application, or currying)
 - Polymorphic types [PJH99]
 - The following function can compute the union of two sets
 - Implemented as (homogeneous) polymorphic Haskell lists
 - Containing elements of any type which (is an instance of class Eq)

```
member :: (Eq a) => a -> [a] -> Bool
member e []      = False
member e (x:xs) = if (e == x) then True else member e xs

union :: (Eq a) => [a] -> [a] -> [a]
union []      ys = ys
union (x:xs)  ys =
    if member x ys then union xs ys else x:union xs ys
```



Declarative prototyping

- Actually, Haskell provides powerful libraries for list and set manipulation (including a library set union function)
- Similarly, Scala provides a library solution for the set union operation, comprising a Set class and a set union method

```
object Test {  
    def main(args: Array[String]) {  
        println(Set(1,2,3,4).union(Set(1,2,7,8,5)))  
        println(Set(1,2,3,4) union Set(1,2,7,8,5))  
    }  
}
```

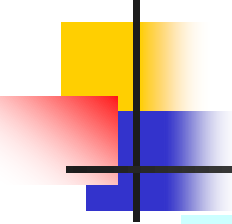
```
> scala Test  
Set(5,1,6,9,2,7,3)  
Set(5,1,6,9,2,7,3)
```



Declarative prototyping

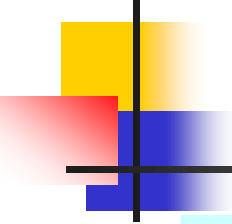
- **Scala provides support for** classes parameterized with types (generic classes) and **pattern matching** (case classes)
 - In the sequel we only consider Scala **immutable (functional) structures**

```
object Test {  
  def member1[T] (e:T,xs:List[T]) : Boolean = xs match {  
    case Nil      => false  
    case (x::xs1) => if (e==x) true else member1(e,xs1)  
  }  
  def union1[T] (xs:List[T],ys:List[T]):List[T] = xs match {  
    case Nil      => ys  
    case (x::xs1) => if (member1(x,ys)) union1(xs1,ys)  
                     else x::union1(xs1,ys)  
  }  
  def main(args: Array[String]) {  
    println(union1(List(6,7,5,3),List(5,6,9,1,2,7)))  
  }  
}  
  
> scala Test  
List(3,5,6,9,1,2,7)
```



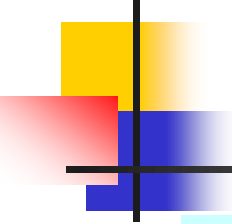
Declarative prototyping

- Both C and Scala (as well C++ or Java) provide support for improved performance based on
 - Mutable (static and dynamic) data structures
 - Imperative style programming
- In the sequel we only consider such options in the imperative C implementations



Declarative prototyping

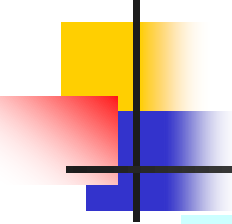
- **Designing the procedural implementation**
 - There are various options:
 - Recursive implementation
 - Implementation as WHILE program
 - Result produced
 - By the normal function return mechanism
 - By using an additional parameter transmitted by reference
 - There are also various options concerning the memory allocation policy
 - Use static structures (arrays)
 - Use dynamic structures (lists)
 - Allocate / not allocate space for the result
 - Alter / not alter the (input) parameters



Declarative prototyping

- We use the following type declaration for the C implementation

```
typedef struct elem {  
    int info;  
    struct elem* next;  
} ELEM, *LIST;
```

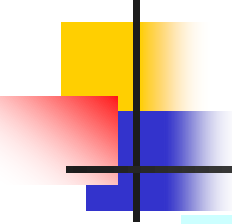


Declarative prototyping

- C implementation of `member`

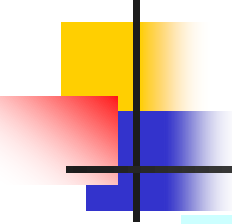
```
typedef enum {false,true} BOOL;
```

```
BOOL member(int e,LIST l)
{
    if (l == 0) return(false);
    else if (e == l->info) return(true);
    else return (member(e,l->next));
}
```



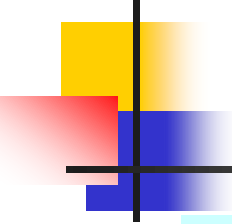
Declarative prototyping

- For `union` we consider four different implementations:
 - The first two variants
 - Alter the input parameters
 - Do not allocate space for the result.
 - The last two variants
 - Do not alter the input parameters
 - Allocate space for the result



Declarative prototyping

```
LIST union(LIST x, LIST y)
{ LIST z;
  if (x == 0) return(y);
  else if (member(x->info, y)) {
    z = union(x->next, y);
    free(x);
    return(z);
  } else {
    z = x;
    z -> next = union(x->next, y);
    return(z);
  }
}
```



Declarative prototyping

- The function can be used as follows:

```
LIST x,y,x;
```

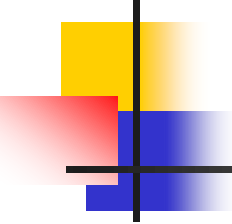
```
...
```

```
/* Create the 'sets' x and y */
```

```
...
```

```
z = union(x,y);
```

```
/* The 'set' z is the union of x and y */
```

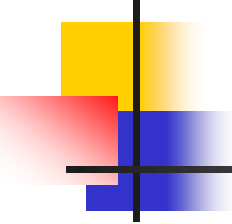


Declarative prototyping

- Alternatively, we can implement `union` as a C function of type `void`; the function returns its result by using an additional parameter transmitted by reference.

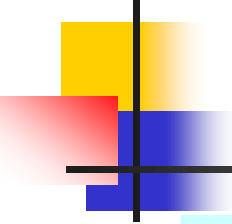
```
void union(LIST x, LIST y, LIST *z)
```

- In the sequel, we find convenient to use the term *procedure* to refer to such a C function of type `void`.



Declarative prototyping

```
void union(LIST x, LIST y, LIST *z)
{
    if (x == 0)    (*z) = y;
    else if (member(x->info, y)) {
        union(x->next, y, z);
        free(x);
    } else {
        (*z) = x;
        union(x->next, y, &((*z)->next));
    }
}
```



Declarative prototyping

- The procedure can be used as follows:

```
LIST x,y,x;
```

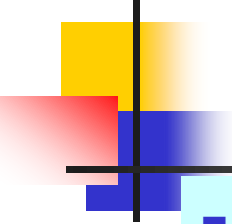
```
...
```

```
/* Create the 'sets' x and y */
```

```
...
```

```
union(x,y,&z);
```

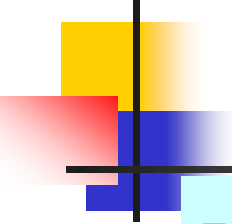
```
/* The 'set' z is the union of x and y */
```



Declarative prototyping

- The C function given below allocates space for the result and does not alter the input parameters.

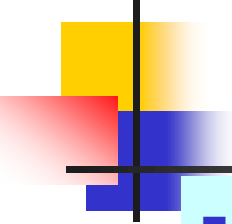
```
LIST union(LIST x, LIST y)
{ LIST z;
  if (x == 0) return(copy(y));
  else if (member(x->info, y)) {
    return (union(x->next, y));
  } else {
    z = (LIST)malloc(sizeof(ELEM));
    z->info = x->info;
    z->next = union(x->next, y);
    return(z);
  }
}
```



Declarative prototyping

- The implementation uses an auxiliary function that makes a physical copy of its parameter.

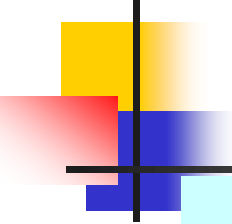
```
LIST copy (LIST l)
{ LIST r;
  if (l == 0) return(0);
  else {
    r = (LIST)malloc(sizeof(ELEM));
    r->info = l->info;
    r->next = copy(l->next);
    return(r);
  }
}
```



Declarative prototyping

- The last implementation solution uses an additional parameter transmitted by reference. It allocates space for the result and does not alter the input parameters.

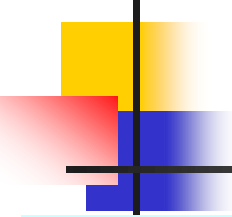
```
void union(LIST x, LIST y, LIST *z)
{
    if (x == 0)    copy(y, z);
    else if (member(x->info, y)) {
        union(x->next, y, z);
    } else {
        (*z) = (LIST)malloc(sizeof(ELEM));
        (*z)->info = x->info;
        union(x->next, y, &((*z)->next));
    }
}
```

Declarative prototyping

- In this case we use the following auxiliary procedure to make a physical copy of a list.

```
void copy(LIST l, LIST *r)
{
    if (l == 0)    (*r)=0;
    else {
        (*r) = (LIST)malloc(sizeof(ELEM));
        (*r)->info = l->info;
        copy(l->next, &((*r)->next));
    }
}
```

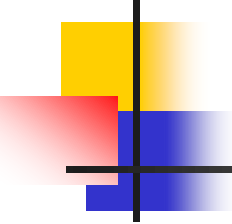


Declarative prototyping

- **Example Merging**
- Haskell specification:

```
merge :: ([Int],[Int]) -> [Int]
merge([],ys) = ys
merge(xs,[]) = xs
merge(x:xs,y:ys) = if (x<y) then x:merge(xs,y:ys)
                  else y:merge(x:xs,ys)
```

- The correctness proof for `merge(xs,ys)` can proceed by induction on the following **computed complexity measure**: `(length(xs) + length(ys))`. The sequences `xs` and `ys` are assumed to be ordered.



Declarative prototyping

- The Haskell prototype behaves as follows:

```
Main> merge ([1,3,5,7], [2,4,6])  
[1,2,3,4,5,6,7]
```



Declarative prototyping

- In Scala you can implement a concrete merge function as follows:

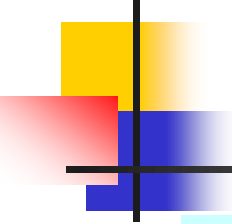
```
object Test {  
  def main(args: Array[String]) {  
    println(merge(List(1,3,5,7),List(2,3,4,6,8)))  
  }  
  def merge (xs:List[Int],ys:List[Int]):List[Int] =  
    (xs,ys) match {  
      case (Nil,ys)           => ys  
      case (xs,Nil)          => xs  
      case (x::xs1,y::ys1) => if (x<y) (x::merge(xs1,y::ys1))  
                             else (y::merge(x::xs1,ys1))  
    }  
}  
  
> scala Test  
List(1,2,3,3,4,5,6,7,8)
```



Declarative prototyping

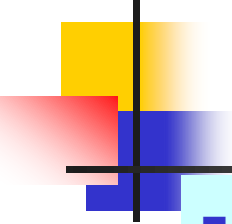
- A **more abstract (generic and) curried version** of the list merging algorithm can be implemented in **Scala** as follows:

```
object Test {  
  def main(args: Array[String]) {  
    println(mergeho(List(1,3,5,7),List(2,3,4,6,8))((x:Int,y:Int) => x<y))  
  }  
  def mergeho[T] (xs:List[T],ys:List[T])(less : (T,T) => Boolean):List[T] =  
    (xs,ys) match {  
      case (Nil,ys)      => ys  
      case (xs,Nil)      => xs  
      case (x::xs1,y::ys1) =>  
        if (less(x,y)) (x::mergeho(xs1,y::ys1)(less))  
        else (y::mergeho(x::xs1,ys1)(less))  
    }  
}  
  
> scala Test  
List(1,2,3,3,4,5,6,7,8)
```



Declarative prototyping

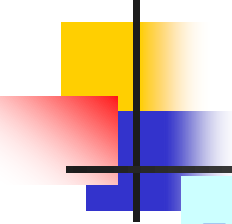
- For `merge` we only design two C implementation solutions (as function / procedure).
- In the both cases the input parameters are altered and no memory is allocated for the result.



Declarative prototyping

- C implementation as a function

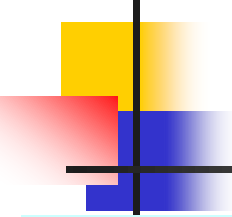
```
LIST merge(LIST x, LIST y)
{ LIST z;
  if (x == 0) return(y);
  else if (y == 0) return(x);
  else if ((x->info) < (y->info)) {
    z=x;
    z->next = merge(x->next, y);
    return(z);
  } else {
    z = y;
    z->next = merge(x, y->next);
    return(z);
  }
}
```



Declarative prototyping

- C implementation as a procedure

```
void merge(LIST x, LIST y, LIST *z)
{
    if (x == 0)    (*z)=y;
    else if (y == 0) (*z)=x;
    else if ((x->info) < (y->info)) {
        (*z)=x;
        merge(x->next, y, &((*z)->next));
    } else {
        (*z)=y;
        merge(x, y->next, &((*z)->next));
    }
}
```

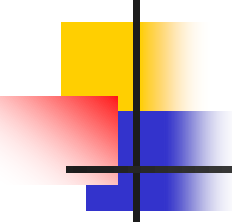



Declarative prototyping

- **Example Tree flattening using difference lists**
- Haskell specification:

```
data Tree = Nil | T(Tree, Int, Tree)
flat(Nil, ys) = ys
flat(T(l, n, r), ys) = flat(l, n:flat(r, ys))
```

- Difference lists notation: if $xs = e1 : \dots : en : ys$ then
 $xs - ys = [e1, \dots, en]$
- The correctness proof for $flat(t, ys)$ can proceed by induction on the structure of t (by **structural induction**).
 $flat(t, ys) - ys =$ the list of nodes in t (obtained by a left-node-right inorder traversal)



Declarative prototyping

- The Haskell prototype behaves as follows:

```
Main> flat(T(T(Nil,2,T(Nil,4,Nil)),1,T(Nil,3,Nil)), [100,100])  
[2,4,1,3,100,100]
```



Declarative prototyping

- In the Scala implementation you can use **case classes** and **pattern matching**

```
object Test {  
  def main(args: Array[String]) {  
    println(flat(T(T(Nil,2,T(Nil,4,Nil)),1,T(Nil,3,Nil)),List(100,100)))  
  }  
  def flat (t:Tree,ys:List[Int]):List[Int] = t match {  
    case Nil => ys  
    case T(l,n,r)    => flat(l,n::flat(r,ys))  
  }  
}  
  
abstract class Tree  
case object Nil extends Tree  
case class T (l:Tree,n:Int,r:Tree) extends Tree  
  
> scala Test  
List(2,4,1,3,100,100)
```



Declarative prototyping

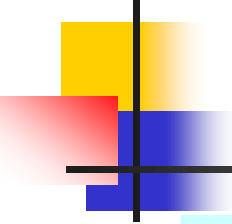
- The flat method can also be allocated to the class Tree

```
object Test {
  def main(args: Array[String]) {
    println((T(T(Nil,2,T(Nil,4,Nil)),1,T(Nil,3,Nil))).flat(List(100,100)))
    println((T(T(Nil,2,T(Nil,4,Nil)),1,T(Nil,3,Nil))) flat List(100,100))
  }
}

class Tree {
  def flat (ys:List[Int]):List[Int] =
    this match {
      case Nil => ys
      case T(l,n,r) => l.flat(n::r.flat(ys))
    }
}

case object Nil extends Tree
case class T (l:Tree,n:Int,r:Tree) extends Tree

> scala Test
List(2,4,1,3,100,100)
List(2,4,1,3,100,100)
```

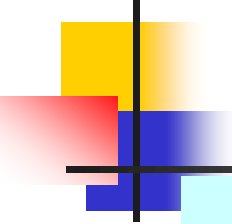


Declarative prototyping

- Apart from the type declaration for lists, in the C implementation we use the following type declaration for trees

```
typedef struct node {  
    int info;  
    struct node *left, *right;  
} NODE, *TREE;
```

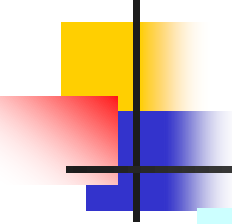
- We offer two implementation solutions.



Declarative prototyping

- C implementation as a function

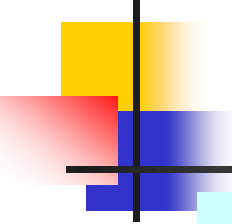
```
LIST flat(TREE t, LIST y)
{ LIST x;
  if (t == 0) return(y);
  else {
    x = (LIST)malloc(sizeof(ELEM));
    x->info = t->info;
    x->next = flat(t->right, y);
    return(flat(t->left, x));
  }
}
```



Declarative prototyping

- C implementation as a procedure

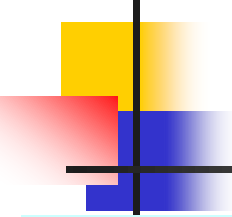
```
void flat(TREE t, LIST *x, LIST y)
{ LIST z;
  if (t == 0) (*x)=y;
  else {
    z = (LIST)malloc(sizeof(ELEM));
    z->info = t->info;
    flat(t->right, &(z->next), y);
    flat(t->left, x, z);
  }
}
```



Declarative prototyping

Rewriting techniques

- Many computations can be described using rewriting techniques.
- Sometimes, a data structure must be prepared before performing some calculations or some transformations on it.
- We want to transform a binary tree in a list.
 - We use a rewriting operation to reduce the complexity of the left sub-tree until it becomes `Nil`.
 - Next, the transformation is applied recursively on the right sub-tree.

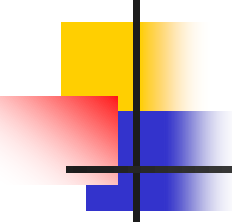


Declarative prototyping

- **Example Tree flattening using a rewriting transformation**
- Haskell specification:

```
data Tree = Nil | T(Tree,Int,Tree)    deriving Show

transf :: Tree -> Tree
transf Nil = Nil
transf (T(Nil,n,r)) = T(Nil,n,transf(r))
transf (T(T(ll,nl,rl),n,r)) =
    transf(T(ll,nl,T(rl,n,r)))
```

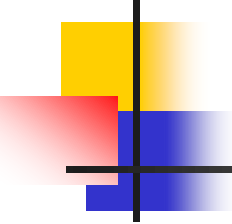


Declarative prototyping

- The Haskell prototype behaves as follows:

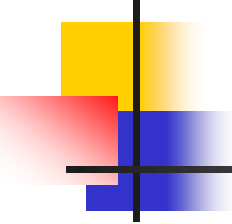
```
Main> transf (T (T (T (Nil, 3, Nil), 2, Nil), 1, Nil))  
T (Nil, 3, T (Nil, 2, T (Nil, 1, Nil)))
```

- The result is a degenerate tree, rather than a list
 - (It is straightforward to design a version of this function with signature: `transf :: Tree -> [Int]`)



Declarative prototyping

- To prove the correctness of `transf` we use a more complex measure.
- The support set is $\mathbf{N} \times \mathbf{N}$, and we use the so-called lexicographic ordering (that we denote here by $\blacktriangleleft$) over $\mathbf{N} \times \mathbf{N}$. The lexicographic ordering is defined as follows:
$$(n_1, m_1) \blacktriangleleft (n_2, m_2) \text{ if } (n_1 < n_2) \text{ or } (n_1 = n_2 \text{ and } m_1 < m_2)$$
- It is easy to check that $\blacktriangleleft$ is a well founded relation over $\mathbf{N} \times \mathbf{N}$. Also, for each $(n, m) \in \mathbf{N} \times \mathbf{N}$ either $(n=0, m=0)$ or $(0,0) \blacktriangleleft (n,m)$.



Declarative prototyping

- The correctness of `transf` can be proved by induction on the following **composed complexity measure**:

$$\begin{aligned} c: \text{Tree} &\rightarrow \mathbf{N} \times \mathbf{N} & u, v: \text{Tree} &\rightarrow \mathbf{N} \\ c(t) &= (u(t), v(t)) & \text{for any } t &:: \text{Tree} \end{aligned}$$

- Here, $u(t)$ is the number of nodes in t and $v(t)$ is a measure of the complexity of the left sub-tree:

$$u(\text{Nil}) = 0$$

$$u(T(l, n, r)) = 1 + u(l) + u(r)$$

$$v(\text{Nil}) = 0$$

$$v(T(l, n, r)) = 1 + v(l)$$

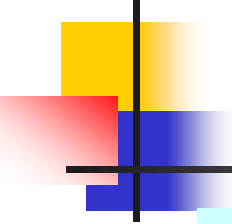
- Remark that $c(t) = (0, 0)$ iff $t = \text{Nil}$.
- We present two different implementation solutions.



Declarative prototyping

■ A Scala implementation

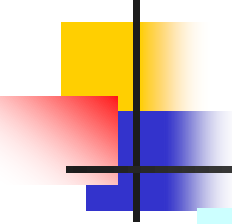
```
object Test {  
  def main(args: Array[String]) {  
    println(transf(T(T(T(Nil,3,Nil),2,Nil),1,Nil)))  
  }  
  def transf (t:Tree):Tree = t match {  
    case Nil                => Nil  
    case T(Nil,n,r)         => T(Nil,n,transf(r))  
    case T(T(l1,nl,r1),n,r) => transf (T(l1,nl,T(r1,n,r)))  
  }  
}  
  
abstract class Tree  
case object Nil extends Tree  
case class T (l:Tree,n:Int,r:Tree) extends Tree  
  
> scala Test  
T(Nil,3,T(Nil,2,T(Nil,1,Nil)))
```



Declarative prototyping

- C implementation as a function

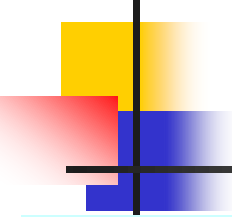
```
TREE transf(TREE t)
{
    TREE p;
    if (t == 0)    return(0);
    else if (t->left == 0){
        t->right = transf(t->right);
        return(t);
    } else {
        p = t;
        t = p->left;  p->left = t->right;
        t->right = p;
        return(transf(t));
    }
}
```



Declarative prototyping

- C procedure with **inout** parameter

```
void transf(TREE *t)
{
    TREE p;
    if ((*t) != 0) {
        if (((*t)->left) == 0)
            transf(&((*t)->right));
        else {
            p = (*t); (*t) = p->left;
            p->left = (*t)->right;
            (*t)->right = p;
            transf(t);
        }
    }
}
```



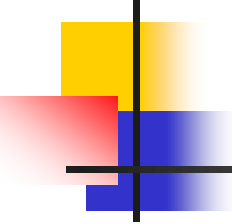
Declarative prototyping

- **Example Mutual recursion and simultaneous induction**
- Haskell specification:

```
data Btree = NilB | B(Int,Ttree,Ttree)
data Ttree = NilT | T(Int,Btree,Btree,Btree)

flatB :: (Btree,[Int]) -> [Int]
flatB (NilB,ys) = ys
flatB (B(n,tl,tr),ys) = n:flatT(tl,flatT(tr,ys))

flatT :: (Ttree,[Int]) -> [Int]
flatT (NilT,ys) = ys
flatT (T(n,b1,bm,br),ys) = n:flatB(b1,flatB(bm,flatB(br,ys)))
```

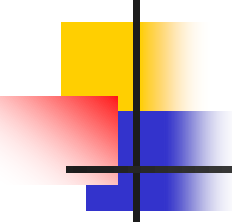
Declarative prototyping

- Let

```
t :: Ttree; b :: Btree
t = T(2,NilB,B(3,NilT,T(4,NilB,NilB,NilB)),NilB)
b = B(1,t,T(5,B(6,NilT,NilT),NilB,B(7,NilT,NilT)))
```

- The Haskell prototype behaves as follows:

```
Main> flatT (t,[0,0,0,0])
[2,3,4,0,0,0,0]
Main> flatB (b,[])
[1,2,3,4,5,6,7]
```



Declarative prototyping

■ Claim

- $\text{flatB}(b, ys) - ys$ = the list of nodes in b (obtained by a node-left-right traversal)
- $\text{flatT}(t, ys) - ys$ = the list of nodes in t (obtained by a node-left-mid-right traversal)

■ **Proof** By **simultaneous induction** on the number of nodes in the tree structure (the first parameter of each function):

- **Base case** For trees with zero nodes the specification is: $\text{flatB}(\text{NilB}, ys) = ys$, $\text{flatT}(\text{NilT}, ys) = ys$; this is correct since $ys - ys = []$. The both functions behave correctly for trees with zero nodes.
- **Induction step** For the induction step each function uses the induction hypothesis of the other function.

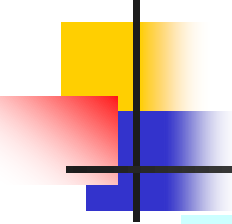


Declarative prototyping

■ Scala implementation with case classes

```
object Test {  
  def main(args: Array[String]) {  
    val t=T(2,NilB,B(3,NilT,T(4,NilB,NilB,NilB)),NilB)  
    val b=B(1,t,T(5,B(6,NilT,NilT),NilB,B(7,NilT,NilT)))  
    println(flatT(t,List(0,0,0,0)))  
    println(flatB(b,List()))  
  }  
  def flatB (b:Btree,ys:List[Int]):List[Int] = b match {  
    case NilB          => ys  
    case B(n,tl,tr)    => n::flatT(tl,flatT(tr,ys))  
  }  
  def flatT (t:Ttree,ys:List[Int]):List[Int] = t match {  
    case NilT          => ys  
    case T(n,bl,bm,br) => n::flatB(bl,flatB(bm,flatB(br,ys)))  
  }  
}  
  
abstract class Btree  
case object NilB extends Btree  
case class B (n:Int,l:Ttree,r:Ttree) extends Btree  
  
abstract class Ttree  
case object NilT extends Ttree  
case class T (n:Int,l:Btree,m:Btree,r:Btree) extends Ttree
```

```
> scala Test  
List(2,3,4,0,0,0,0)  
List(1,2,3,4,5,6,7)
```



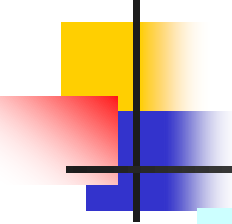
Declarative prototyping

- For the C implementation we use the following type declarations:

```
typedef struct Bnode {  
    int info;  
    struct Tnode *l, *r;  
} BNODE, *BTREE;
```

```
typedef struct Tnode {  
    int info;  
    struct Bnode *l, *m, *r;  
} TNODE, *TTREE;
```

- We give implementations as functions and procedures.

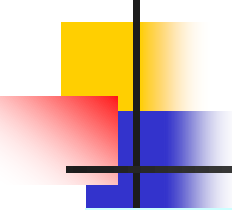


Declarative prototyping

- A pair of functions

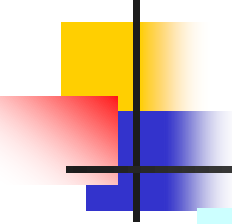
```
LIST flatT(TTREE, LIST);
```

```
LIST flatB(BTREE b, LIST y)
{ LIST x;
  if (b == 0) return(y);
  else {
    x = (LIST)malloc(sizeof(ELEM));
    x->info = b->info;
    x->next = flatT(b->l, flatT(b->r, y));
    return(x);
  }
}
```



Declarative prototyping

```
LIST flatT(TTREE t,LIST y)
{ LIST x;
  if (t == 0) return(y);
  else {
    x = (LIST)malloc(sizeof(ELEM));
    x->info = t->info;
    x->next = flatB(t->l,flatB(t->m,flatB(t->r,y)));
    return(x);
  }
}
```

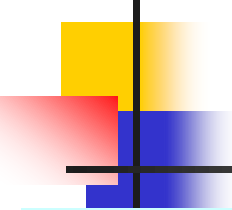


Declarative prototyping

- A pair of procedures

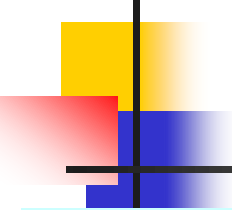
```
void flatT(TTREE, LIST *, LIST);
```

```
void flatB(BTREE b, LIST *x, LIST y)
{ LIST z;
  if (b == 0) (*x)=y;
  else {
    (*x) = (LIST)malloc(sizeof(ELEM));
    (*x)->info = b->info;
    flatT(b->r, &z, y);
    flatT(b->l, &((*x)->next), z);
  }
}
```



Declarative prototyping

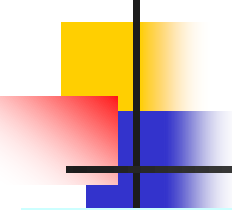
```
void flatT(TTREE t,LIST *x,LIST y)
{ LIST z,w;
  if (t == 0) (*x) = y;
  else {
    (*x) = (LIST)malloc(sizeof(ELEM));
    (*x)->info = t->info;
    flatB(t->r,&z,y);
    flatB(t->m,&w,z);
    flatB(t->l,&((*x)->next),w);
  }
}
```

Declarative prototyping

- **Remark** In C you may need to employ unions in order to implement Haskell user-defined types with multiple variants.
- **Example** Haskell type:

```
data Lisp = Nil | Atom Int | Cons (Lisp,Lisp)
```



Declarative prototyping

- The above Haskell definition can be implemented in C as follows:

```
typedef enum {atom,cons} SEL;
typedef struct Lisp {
    SEL sel;    // selector field
    union {
        int atom;
        struct {
            struct Lisp *car;
            struct Lisp *cdr;
        } cons;
    } lisp;
} CEL, *LISP;
```



Declarative prototyping

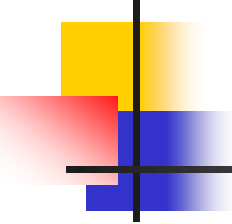
- In the Scala implementation you can use the Composite design pattern

```
abstract class Lisp
case object Nil extends Lisp
case class Atom (n:Int) extends Lisp
case class Cons (car:Lisp,cdr:Lisp) extends Lisp
```

- You can make the following experiment:

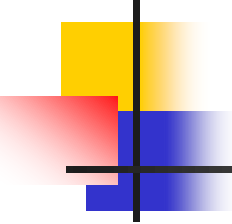
```
object Test {
  def main(args: Array[String]) {
    println(Cons(Cons(Atom(1),Cons(Atom(2),Nil)),Atom(10)))
  }
}

> scala Test
Cons(Cons(Atom(1),Cons(Atom(2),Nil)),Atom(10))
```



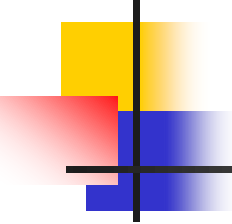
References

- [Boe84] B. Boehm, et al, Prototyping versus specifying: a multi-project experiment, *IEEE Transactions on Software Engineering*, vol. 10(3), pp. 290-303, 1984.
- [CF58] H. Curry, R. Feys. *Combinatory logic*, North Holland, 1958.
- [Mit96] J.C. Mitchell, *Foundations for programming languages*, MIT Press, 1996.



References

- [Mur96] T. Muresan, Software Engineering – Lecture Notes, Technical University of Cluj-Napoca, 1996.
- [OSV16] M. Odersky, L. Spoon, B. Venners, *Programming in Scala (Third Edition)*, Artima, 2016.
- [PJH99] S. Peyton-Jones, R.J.M. Hughes (eds), Report on the programming language Haskell 98; available at <http://www.haskell.org>, 1999.



References

- [Pie18] B. Pierce, *Software Foundations*, vol. 1 (*Logical Foundations*), 2018; available at <https://softwarefoundations.cis.upenn.edu/>
- [Som15] I. Sommerville, *Software Engineering*, (10th edition), Addison-Wesley, 2015.
- [Zav89] P. Zave, A compositional approach to multiparadigm programming, *IEEE Software*, vol. 6(5), pp. 15-27, 1989.